# Structured Use Cases OMG Proposal Draft v0.5

Doug Rosenberg – Parallel Agile, Inc.

www.parallelagile.com

## Table of Contents

# 1. Overview

**Structured Use Cases is a proposed alternative use case library for SysML v2. The proposal consists of two complementary parts: a Structured Use Case Specification Library and a Structured Use Case Diagram Library.**

SysML v2 implements use case modeling through a standard library. Because the language is designed to evolve through its libraries, this proposal takes the form of an alternative use case library rather than a language revision. The intent is to improve usability, interoperability, and standardization while remaining compatible with the overall SysML v2 architecture. The increased emphasis on alternate and exception scenarios promotes more complete discovery of off-nominal requirements while improving behavioral test coverage.

The proposal standardizes structured use case specifications while restoring a simple and natural approach to creating use case diagrams. Together these libraries improve the value-to-pain ratio of use case modeling without requiring changes to the SysML v2 language itself.

## 2. Background

The modeling philosophy presented in this paper has evolved over several decades through industrial practice, consulting, commercial tool development, teaching, and published research, including several books:

- **Use Case Driven Object Modeling with UML: A Practical Approach** (1999)
- **Applying Use Case Driven Object Modeling with UML: An Annotated eCommerce Example** (2001)
- **Use Case Driven Object Modeling with UML: Theory and Practice** (2007)
- **Design Driven Testing: Test Smarter, Not Harder** (2010)

This proposal extends that body of work by standardizing both structured use case specifications and simplified use case diagrams.

Structured use case specifications have been successfully used in industry for many years. For example, the **Sparx Systems Structured Scenario Editor** (Figure 1), introduced in conjunction with the **Design Driven Testing** book, demonstrated the practical value of organizing behavior into basic, alternate, and exception scenarios composed of individual steps.

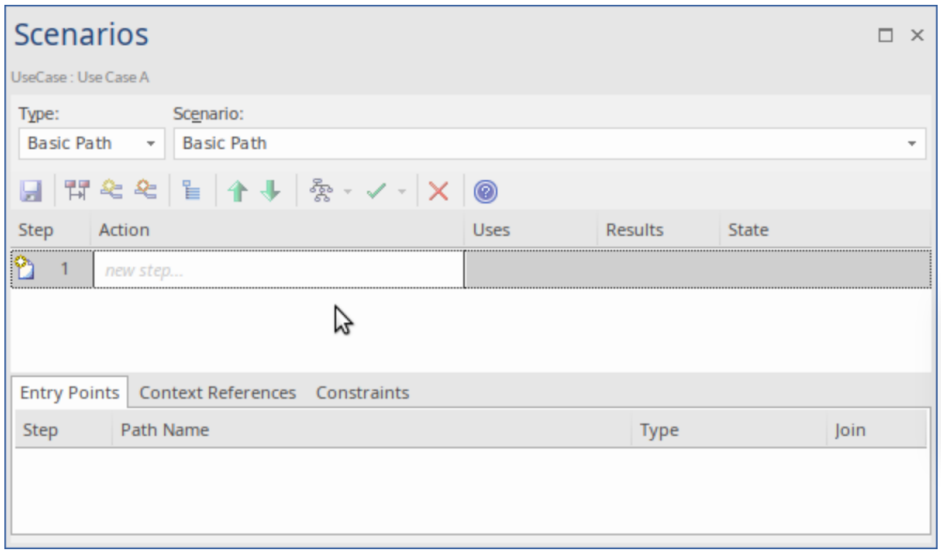



*Figure 1 – Use Case Modelers have worked with scenarios for decades*

More than fifteen years later, this methodology remains in active industrial use. Expressing use cases as a set of scenarios which consist of steps is much more aligned with current practice than is the existing SysML v2 use case library.

## 3. Use Case Modeling Philosophy

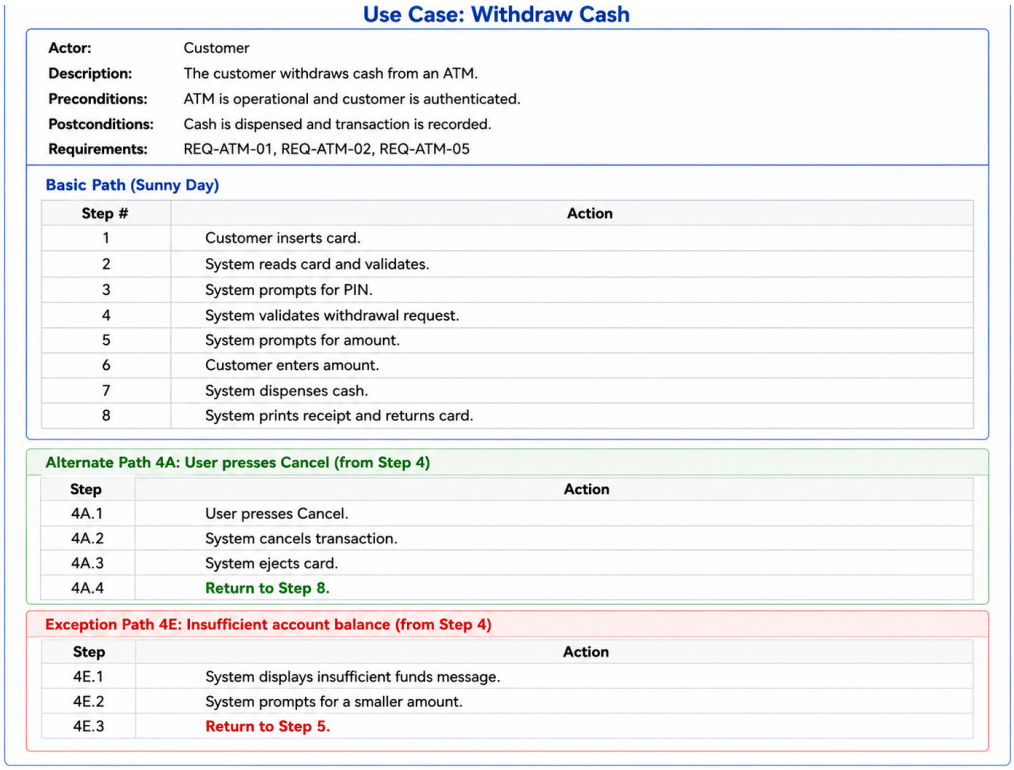
**KEY PRINCIPLE**

**The primary engineering value of use case modeling resides in the use case specifications, not in the use case diagrams.**

Use case diagrams provide a simple overview of system behavior by organizing system functionality into individual use cases and showing the relationships among those use cases and their actors. Their primary purpose is to help readers understand the functional organization of a system and navigate quickly to the corresponding use case specifications.

Because every system is different, use case diagrams should provide sufficient flexibility to organize and communicate functionality effectively. They should not be overly restrictive. The detailed semantics belong primarily in the accompanying use case specifications, while the diagram serves as an intuitive organizational and communication aid.

A use case specification captures the behavior associated with a single use case through scenarios and ordered steps.



**Use Case: Withdraw Cash**

| | |
|---|---|
| **Actor:** | Customer |
| **Description:** | The customer withdraws cash from an ATM. |
| **Preconditions:** | ATM is operational and customer is authenticated. |
| **Postconditions:** | Cash is dispensed and transaction is recorded. |
| **Requirements:** | REQ-ATM-01, REQ-ATM-02, REQ-ATM-05 |

**Basic Path (Sunny Day)**

| Step # | Action |
|---|---|
| 1 | Customer inserts card. |
| 2 | System reads card and validates. |
| 3 | System prompts for PIN. |
| 4 | System validates withdrawal request. |
| 5 | System prompts for amount. |
| 6 | Customer enters amount. |
| 7 | System dispenses cash. |
| 8 | System prints receipt and returns card. |

**Alternate Path 4A: User presses Cancel (from Step 4)**

| Step | Action |
|---|---|
| 4A.1 | User presses Cancel. |
| 4A.2 | System cancels transaction. |
| 4A.3 | System ejects card. |
| 4A.4 | **Return to Step 8.** |

**Exception Path 4E: Insufficient account balance (from Step 4)**

| Step | Action |
|---|---|
| 4E.1 | System displays insufficient funds message. |
| 4E.2 | System prompts for a smaller amount. |
| 4E.3 | **Return to Step 5.** |

*Figure 2. Example Use Case Specification*

To reiterate, the primary purpose of use case modeling is not merely to document known requirements, but to help discover missing requirements. The scenarios in a use case specification, particularly the alternate and exception scenarios, are where many off-nominal requirements are discovered. This is why the specification, rather than the diagram, contains the primary engineering value of the use case model: it is where requirements are discovered, system behavior is defined, and comprehensive behavioral tests can ultimately be derived.

## 3.1 Relationship Between Structured Scenarios and Activity Diagrams

Structured scenarios and activity diagrams are complementary modeling techniques that serve different purposes during systems engineering. Structured scenarios are particularly effective during requirements elicitation, while activity diagrams excel at refining and communicating the execution of those requirements.

A common misperception is that activity diagrams can serve as a complete replacement for structured use case scenarios. While activity diagrams are excellent for modeling execution behavior, they do not naturally guide analysts through the systematic exploration of alternate and exception behavior that is central to effective requirements elicitation. As a result, important behavioral requirements may never be discovered, even though the notation is fully capable of expressing them.

The difference lies less in the expressive power of the notations than in the way they guide the analyst's thinking.

A structured scenario approach provides more than a documentation format. It guides analysts through a systematic process of behavioral discovery. By explicitly documenting the Basic Scenario together with Alternate and Exception Scenarios, analysts are continually encouraged to ask:

- What must be true for this step to succeed?
- What could prevent this step from succeeding?
- What alternate choices could the actor make?
- How should the system respond when something unexpected occurs?
- Does execution rejoin the Basic Scenario, or does the use case end?

This structured thought process naturally uncovers prerequisite behaviors, validations, alternate outcomes, recovery paths, and exceptional conditions. Considering alternate and exception scenarios frequently improves the quality of the Basic Scenario itself. For example, asking what happens when an ATM customer has insufficient funds naturally reveals that the nominal behavior must first include a step in which the banking system verifies that sufficient funds are available. The exploration of off-nominal behavior often exposes missing requirements in the nominal behavior.

By contrast, when use case behavior is modeled directly as an activity diagram, the analyst's attention naturally gravitates toward the nominal execution path. A straightforward activity diagram progressing from start to finish often appears complete because there is a visible path through the system. Under schedule pressure, teams frequently do not go beyond the primary flow, leaving alternate and exception behavior unexplored or undocumented. The issue is not that activity diagrams cannot represent this behavior—they certainly can—but that the modeling process does not continually encourage analysts to discover it.

As additional alternate paths, exception paths, synchronization, timing constraints, events, and concurrent behavior are incorporated, activity diagrams naturally become larger and more interconnected. This additional detail is valuable once the required behavior has been discovered, but it can make activity diagrams less effective as the primary vehicle for eliciting behavioral requirements.

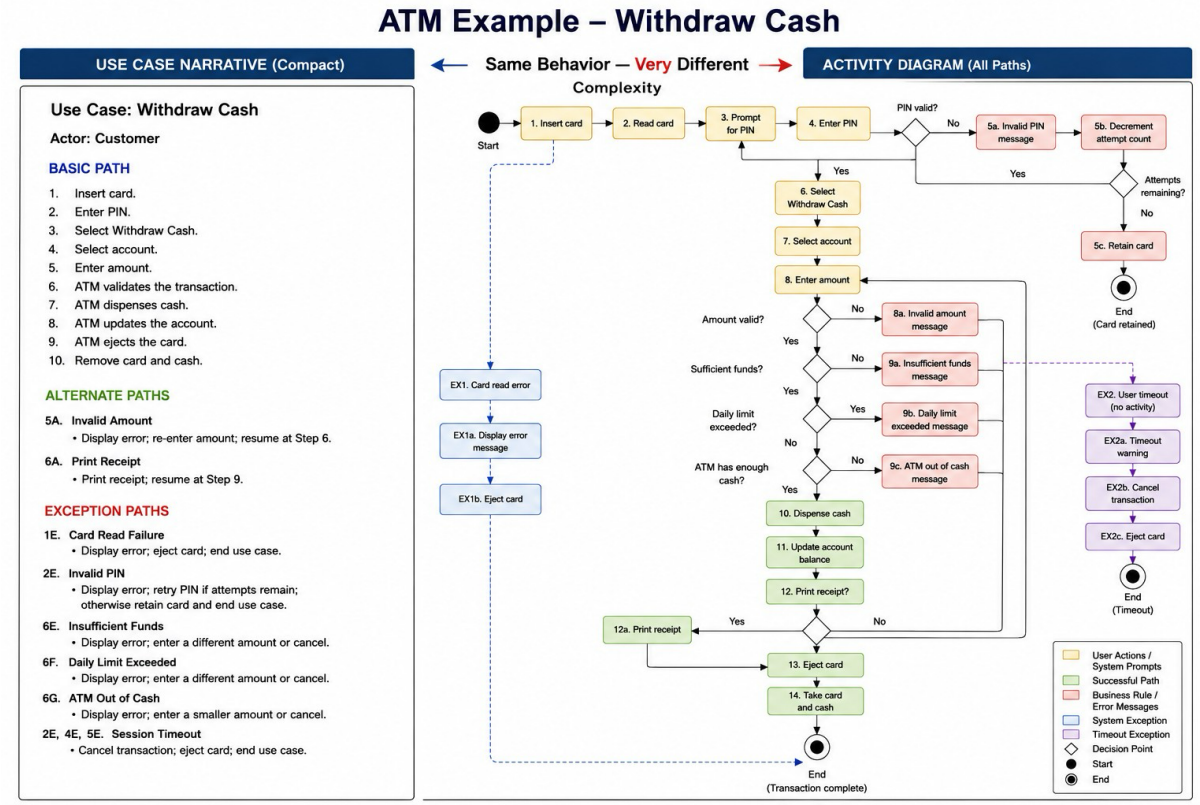



**Figure 3 – Activity diagrams become cumbersome when off-nominal behavior is included.**

Structured scenarios and activity diagrams therefore serve complementary roles within the modeling process.

Structured scenarios guide the discovery and organization of behavioral requirements by continually focusing attention on off-nominal behavior. Activity diagrams refine and communicate the execution semantics of those requirements, including timing, events, asynchronous behavior, concurrency, synchronization, and control flow.

For this reason, this specification treats structured scenarios as the primary mechanism for capturing use case behavior, while recognizing activity diagrams as an important complementary technique for elaborating and communicating the dynamic execution of that behavior.

# 4. The Structured Use Cases Library

## 4.1 Overview

The Structured Use Cases library provides a compact SysML v2 representation for structured use case specifications and flexible use case diagrams. Its central purpose is to standardize the engineering artifact that carries the most behavioral value in use case modeling: the structured use case specification, organized into basic, alternate, and exception scenarios composed of ordered steps.

The library also restores a simple and familiar use case diagram model. Structured Use Cases permits associations on use case diagrams without imposing unnecessary restrictions on what an association may connect. A modeler can connect an actor to a use case, or connect two use cases together in the familiar style used for relationships such as include and extend. Benefits of this relaxed diagram model include making it easy to connect one actor to multiple use cases and making it possible to use relationships such as «precede» to indicate that one use case happens before another.

The updated library is intentionally small. It defines the minimum structure needed to represent actors, use cases, scenarios, steps, associations, and diagrams while leaving domain-specific relationship semantics open for refinement by projects, profiles, tools, or later standardization work.

This section includes the Structured Use Cases library code itself, two test cases that exercise the library, an ATM example expressed in UCML, and a test procedure for loading the library and test cases into Cameo Systems Modeler. The first test case is a simple smoke test. The second is a more realistic test based on an ATM machine.

## 4.2 Library Design

### 4.2.1 Structured Specification Semantics

A UseCase contains one optional basic scenario and zero or more alternate and exception scenarios. Each Scenario contains zero or more Steps and may define its own postcondition. Scenario-specific postconditions are important because alternate and exception flows often terminate in different valid system states.

Each Step has a string stepNumber and a textual description. The string identifier supports ordinary numeric steps and off-nominal step identifiers such as 4A.1 or 5E.2 without imposing a numeric-only ordering scheme.

The library also restores associations as a natural part of use case diagrams. Structured Use Cases permits associations without imposing unnecessary restrictions on what an association may connect. This supports the common actor-to-use-case relationship and also supports use-case-to-use-case relationships familiar to practitioners who have used include and extend. In the SysML v2 library definition below, Actor and UseCase specialize Endpoint, and Association

references two Endpoints; this gives the diagram library its intended flexibility while keeping the detailed behavioral semantics in the structured use case specifications.

## 4.3 StructuredUseCases Library Code

The following SysML v2 textual notation is the current endpoint-based draft of the StructuredUseCases library.

Listing 1. StructuredUseCases_endpoint.sysml

```
library package StructuredUseCases {

    private import ScalarValues::*;
    private import SysML::*;


    // ------------------------------------------------------------
    // Diagram endpoints
    // ------------------------------------------------------------

    part def Endpoint;


    part def Actor :> Endpoint {
        attribute name : String;
        attribute description : String [0..1];
    }


    // ------------------------------------------------------------
    // Structured use case specification
    // ------------------------------------------------------------

    part def Step {
        attribute stepNumber : String;
        attribute description : String;
    }


    part def Scenario {
        attribute name : String;
        attribute description : String [0..1];
        attribute postcondition : String [0..1];

        part step : Step [0..*];
    }


    part def UseCase :> Endpoint {
        attribute name : String;
        attribute description : String [0..1];

        part basicScenario : Scenario [0..1];
        part alternateScenario : Scenario [0..*];
        part exceptionScenario : Scenario [0..*];
    }


    // ------------------------------------------------------------
    // Diagram associations
    // ------------------------------------------------------------
```

```
    part def Association {
        ref part end1 : Endpoint;
        ref part end2 : Endpoint;
    }


    // ------------------------------------------------------------
    // Use case diagram
    // ------------------------------------------------------------

    part def UseCaseDiagram {
        attribute name : String;

        part diagramActor : Actor [0..*];
        part useCase : UseCase [0..*];
        part association : Association [0..*];
    }


    part useCaseModel : UseCaseDiagram;
}
```

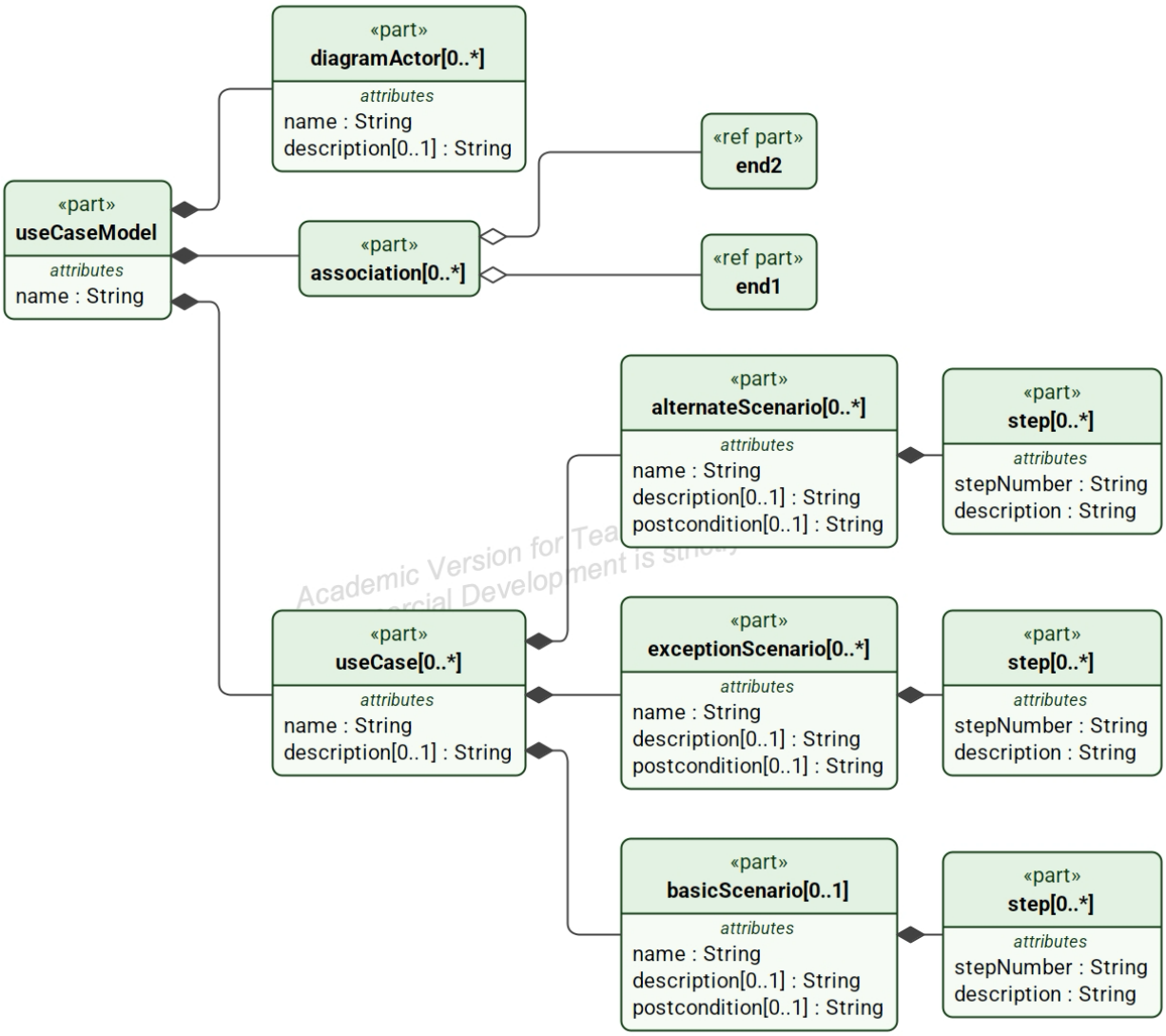



Figure 4. Generated structural view of the StructuredUseCases library.

## 4.4 Smoke Test

The smoke test verifies that the core library types resolve, that multiple endpoint types can coexist, that endpoint associations accept actors and use cases, and that string step identifiers compile as expected.

Listing 2. StructuredUseCases_SmokeTest_endpoint.sysml

```
package StructuredUseCases_SmokeTest {

    private import StructuredUseCases::*;
```

```
    // TEST 1 — Core definitions resolve

    part testEndpoint : Endpoint;
    part testActor : Actor;
    part testStep : Step;
    part testScenario : Scenario;
    part testUseCase : UseCase;
    part testAssociation : Association;
    part testUseCaseDiagram : UseCaseDiagram;


    // TEST 2 — Multiple endpoint types can coexist

    part actorEndpoint : Actor;
    part useCaseEndpoint : UseCase;


    // TEST 3 — Association accepts Actor and UseCase endpoints

    part actorToUseCase : Association {
        ref part end1 = actorEndpoint;
        ref part end2 = useCaseEndpoint;
    }


    // TEST 4 — Association is intentionally permissive:
    // Actor-to-Actor is allowed because both are Endpoints

    part actorEndpoint2 : Actor;

    part actorToActor : Association {
        ref part end1 = actorEndpoint;
        ref part end2 = actorEndpoint2;
    }


    // TEST 5 — UseCase-to-UseCase is also allowed

    part useCaseEndpoint2 : UseCase;

    part useCaseToUseCase : Association {
        ref part end1 = useCaseEndpoint;
        ref part end2 = useCaseEndpoint2;
    }


    // TEST 6 — String step identifiers resolve

    part stringStep : Step {
        attribute stepNumber = "4A.1";
        attribute description =
            "Smoke-test step using a structured string identifier.";
    }


    // TEST 7 — More than one diagram instance can exist

    part secondUseCaseDiagram : UseCaseDiagram;
}
```

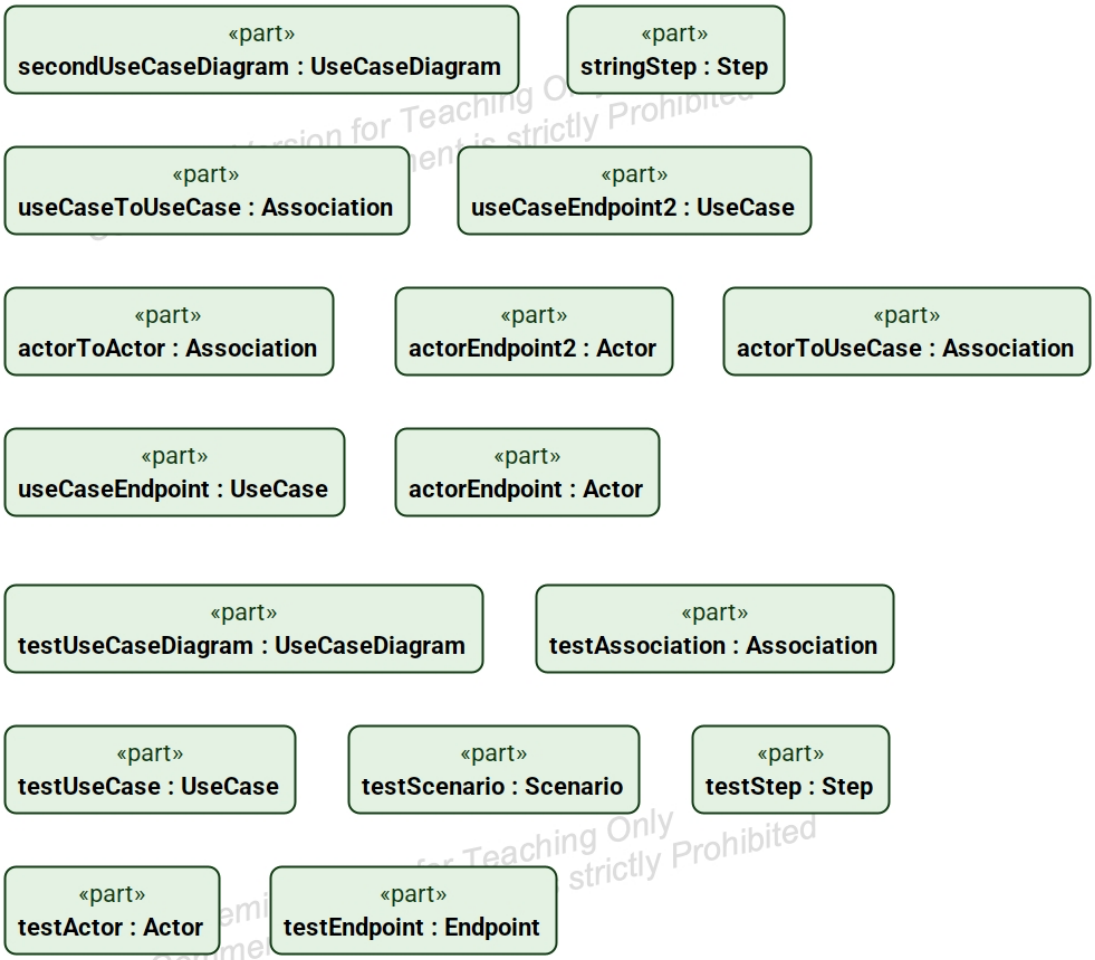



Figure 5. Generated structural view of the smoke test model.

## 4.5 ATM Example Test

The ATM example exercises the library with a realistic use case. It defines an ATM customer actor, a Withdraw Cash use case, a successful basic scenario, an alternate scenario for requesting a different amount, and an exception scenario for insufficient funds. It also includes endpoint-based associations from an actor to a use case and from one use case to another.

Listing 3. StructuredUseCases_ATM_Test_endpoint.sysml

```
package StructuredUseCases_ATM_Test {

    private import StructuredUseCases::*;


    // ============================================================
    // ACTOR
    // ============================================================
```

```
    part def ATMCustomer :> Actor {

        attribute :>> name = "ATM Customer";
        attribute :>> description =
            "Bank customer using the ATM to withdraw cash.";
    }


    // ============================================================
    // BASIC SCENARIO
    // ============================================================

    part def SuccessfulWithdrawal :> Scenario {

        attribute :>> name = "Successful Cash Withdrawal";
        attribute :>> description =
            "Customer withdraws cash using a valid card and sufficient
funds.";

        attribute :>> postcondition =
            "Cash is dispensed, the account is debited, and the card is
returned.";

        part insertCard : Step {
            attribute stepNumber = "1";
            attribute description =
                "Customer inserts a valid bank card.";
        }

        part enterPIN : Step {
            attribute stepNumber = "2";
            attribute description =
                "ATM prompts for the PIN and the customer enters it.";
        }

        part selectWithdrawal : Step {
            attribute stepNumber = "3";
            attribute description =
                "Customer selects Withdraw Cash.";
        }

        part enterAmount : Step {
            attribute stepNumber = "4";
            attribute description =
                "Customer enters the requested withdrawal amount.";
        }

        part authorizeWithdrawal : Step {
            attribute stepNumber = "5";
            attribute description =
                "ATM verifies that the account has sufficient available
funds.";
        }

        part dispenseCash : Step {
            attribute stepNumber = "6";
            attribute description =
                "ATM dispenses the requested cash.";
        }

        part completeTransaction : Step {
            attribute stepNumber = "7";
```

```
            attribute description =
                "ATM debits the account and returns the customer's card.";
        }
    }


    // ============================================================
    // ALTERNATE SCENARIO
    // ============================================================

    part def RequestDifferentAmount :> Scenario {

        attribute :>> name = "Request Different Amount";
        attribute :>> description =
            "ATM cannot dispense the requested denomination and asks the
customer for another amount.";

        attribute :>> postcondition =
            "Customer enters a different withdrawal amount and withdrawal
processing continues.";

        part amountUnavailable : Step {
            attribute stepNumber = "4A";
            attribute description =
                "ATM determines that the requested amount cannot be
dispensed with available denominations.";
        }

        part notifyCustomer : Step {
            attribute stepNumber = "4A.1";
            attribute description =
                "ATM informs the customer that the requested amount cannot
be dispensed.";
        }

        part requestNewAmount : Step {
            attribute stepNumber = "4A.2";
            attribute description =
                "Customer enters a different withdrawal amount.";
        }
    }


    // ============================================================
    // EXCEPTION SCENARIO
    // ============================================================

    part def InsufficientFunds :> Scenario {

        attribute :>> name = "Insufficient Funds";
        attribute :>> description =
            "The customer's account does not contain sufficient available
funds.";

        attribute :>> postcondition =
            "No cash is dispensed and the account balance is unchanged.";

        part detectInsufficientFunds : Step {
            attribute stepNumber = "5E";
            attribute description =
                "ATM determines that available funds are less than the
requested amount.";
        }
```

```
        part displayMessage : Step {
            attribute stepNumber = "5E.1";
            attribute description =
                "ATM informs the customer that sufficient funds are not
available.";
        }

        part cancelWithdrawal : Step {
            attribute stepNumber = "5E.2";
            attribute description =
                "ATM cancels the withdrawal and returns to transaction
selection.";
        }
    }


    // ================================================================
    // USE CASE
    // ================================================================

    part def WithdrawCash :> UseCase {

        attribute :>> name = "Withdraw Cash";

        attribute :>> description =
            "ATM Customer withdraws cash from a bank account.";

        part successfulWithdrawal : SuccessfulWithdrawal;
        part requestDifferentAmount : RequestDifferentAmount;
        part insufficientFunds : InsufficientFunds;
    }


    // ================================================================
    // TEST INSTANCES
    // ================================================================

    part atmCustomer : ATMCustomer;
    part withdrawCash : WithdrawCash;


    // ================================================================
    // ENDPOINT-BASED ASSOCIATION
    // ================================================================

    part customerWithdrawCash : Association {
        ref part end1 = atmCustomer;
        ref part end2 = withdrawCash;
    }


    // ================================================================
    // PERMISSIVENESS CHECK
    // A second use case can also be connected directly to a use case.
    // ================================================================

    part def CheckBalance :> UseCase {
        attribute :>> name = "Check Balance";
        attribute :>> description =
            "ATM Customer views the current account balance.";
    }
```

```
    part checkBalance : CheckBalance;

    part withdrawCashToCheckBalance : Association {
        ref part end1 = withdrawCash;
        ref part end2 = checkBalance;
    }
}
```

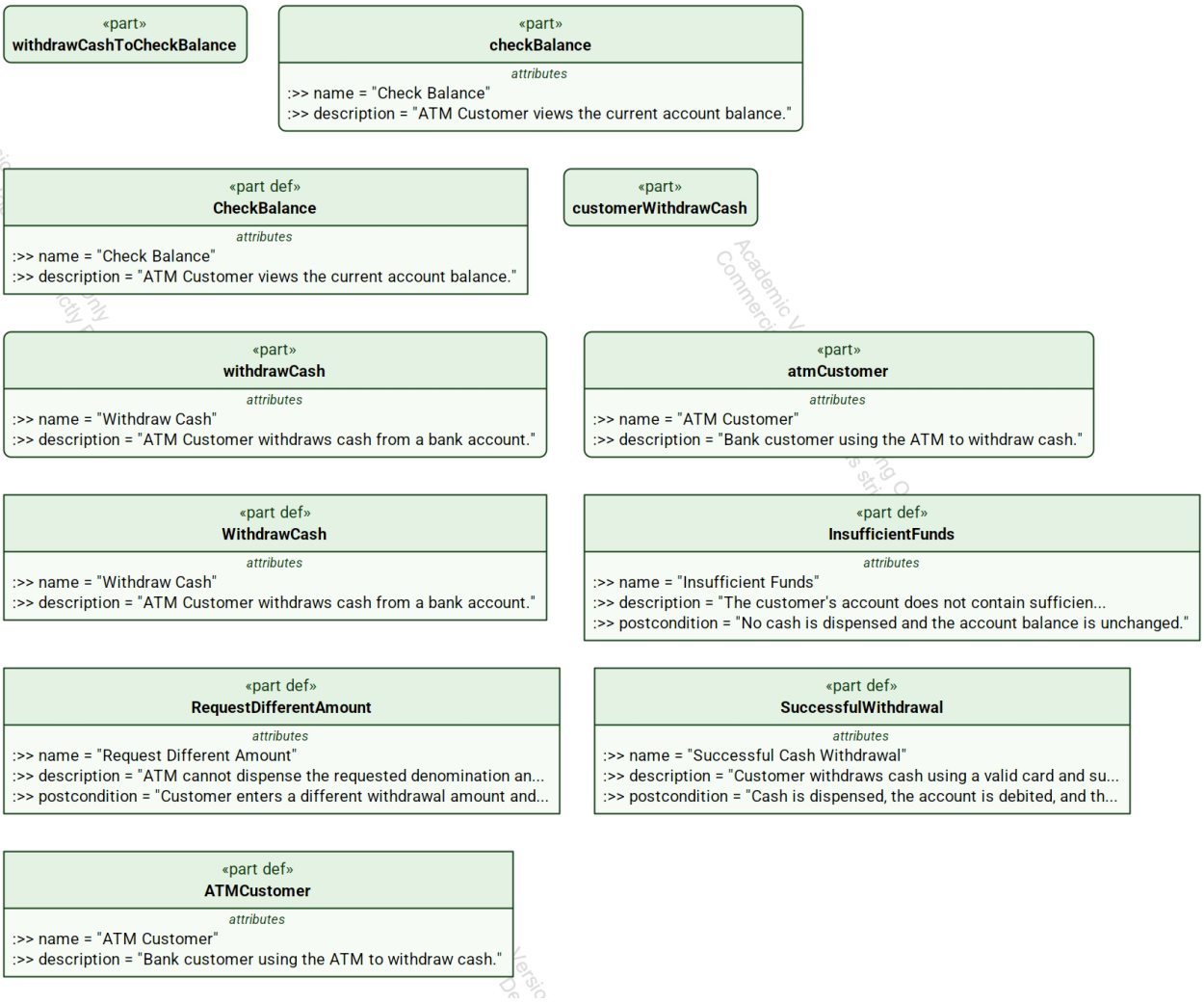



Figure 6. Generated structural view of the ATM example test model.

## 4.6 UCML Example Alignment

The UCML example expresses the same modeling style in a concise textual notation intended for the reference editor and interchange workflows. It is included here as supporting context rather than as the normative SysML v2 library definition. See Section 5 for more detail on UCML.

Listing 4. ATM use case in UCML

```
diagram ATMUseCaseModel {

    actor ATMCustomer {
        name "ATM Customer"
        description "Bank customer using the ATM to withdraw cash."
    }


    usecase WithdrawCash {

        name "Withdraw Cash"
        description "ATM Customer withdraws cash from a bank account."


        basic SuccessfulWithdrawal {

            name "Successful Cash Withdrawal"

            description
                "Customer withdraws cash using a valid card and sufficient
funds."

            step "1"
                "Customer inserts a valid bank card."

            step "2"
                "ATM prompts for the PIN and the customer enters it."

            step "3"
                "Customer selects Withdraw Cash."

            step "4"
                "Customer enters the requested withdrawal amount."

            step "5"
                "ATM verifies that the account has sufficient available
funds."

            step "6"
                "ATM dispenses the requested cash."

            step "7"
                "ATM debits the account and returns the customer's card."

            postcondition
                "Cash is dispensed, the account is debited, and the card is
returned."
        }


        alternate RequestDifferentAmount {

            name "Request Different Amount"

            description
                "ATM cannot dispense the requested denomination and asks
the customer for another amount."

            step "4A"
                "ATM determines that the requested amount cannot be
dispensed with available denominations."
```

```
            step "4A.1"
                "ATM informs the customer that the requested amount cannot
be dispensed."

            step "4A.2"
                "Customer enters a different withdrawal amount."

            postcondition
                "Customer enters a different withdrawal amount and
withdrawal processing continues."
        }


        exception InsufficientFunds {

            name "Insufficient Funds"

            description
                "The customer's account does not contain sufficient
available funds."

            step "5E"
                "ATM determines that available funds are less than the
requested amount."

            step "5E.1"
                "ATM informs the customer that sufficient funds are not
available."

            step "5E.2"
                "ATM cancels the withdrawal and returns to transaction
selection."

            postcondition
                "No cash is dispensed and the account balance is
unchanged."
        }
    }


    association ATMCustomer -> WithdrawCash
}
```

## 4.7 Cameo Test Procedure

Purpose. Verify that the StructuredUseCases library loads, compiles, synchronizes, and resolves all library and test elements in Cameo with the SysML v2 plugin installed.

1. Open Cameo with the SysML v2 plugin installed.
2. Create or open a clean SysML v2 test project.
3. Load StructuredUseCases_endpoint.sysml in the SysML v2 Textual Editor.
4. Synchronize the library using Alt+S.
5. Verify that the library compiles without errors.
6. In the Containment tree, verify that StructuredUseCases contains Actor, Step, Scenario, UseCase, Association, UseCaseDiagram, and useCaseModel : UseCaseDiagram.

7. Do not proceed until the library itself passes.
8. Load StructuredUseCases_SmokeTest_endpoint.sysml in the SysML v2 Textual Editor.
9. Synchronize the smoke test using Alt+S.
10. Verify that the smoke test compiles without errors and that Actor, Step, Scenario, UseCase, Association, Endpoint, and UseCaseDiagram usages resolve.
11. Verify that Actor-to-UseCase, Actor-to-Actor, and UseCase-to-UseCase associations are accepted as endpoint-based associations.
12. Load StructuredUseCases_ATM_Test_endpoint.sysml in the SysML v2 Textual Editor.
13. Synchronize the ATM test using Alt+S.
14. Verify that the ATM actor, use cases, scenarios, steps, and associations compile and resolve.
15. Record PASS if the library, smoke test, and ATM test compile, synchronize, and resolve all expected types correctly.
16. Record FAIL if Cameo reports a compilation, synchronization, import, or type-resolution error.

Important rule. Do not change the library merely to make a test pass. If a failure appears, first investigate package naming, imports, endpoint typing, and the test harness.

# 5. Use Case Modeling Language (UCML)

UCML consists of a textual specification language and a graphical diagram language. The specification captures the behavioral semantics of a use case, while the diagram provides a concise visual representation suitable for communication among stakeholders. The textual notation intentionally resembles the style of SysML v2 by using a keyword-oriented, block-structured format while remaining an independent language. The grammar presented in this section is conceptual rather than parser-oriented; its purpose is to define the major modeling concepts and their relationships. The graphical notation and textual notation are complementary representations of the same underlying model.

## 5.1 Specification Grammar

```
UseCaseSpecification
    ::= "use case" Name
        LinkedRequirements*
        LinkedTestCases*
        Precondition
        Scenario+

Scenario
    ::= ScenarioType
        Step+
        Postcondition
        RejoinStep*

ScenarioType
    ::= "basic"
      | "alternate"
      | "exception"

Step
    ::= StepIdentifier
        StepText

LinkedRequirements
    ::= Requirement*

LinkedTestCases
    ::= TestCase*

Precondition
    ::= Condition

Postcondition
    ::= Condition

RejoinStep
    ::= StepReference
```

## 5.2 Diagram Grammar

```
UseCaseDiagram
    ::= Node*
```

```
        Association*

Node
    ::= Actor
      | UseCase

Association
    ::= AssociationType
        SourceNode
        TargetNode

AssociationType
    ::= "participates"
      | "include"
      | "extend"
      | "generalization"
      | "invokes"
      | "precedes"
```

The following figures illustrate the same Withdraw Cash use case in graphical and specification-editor form. The complete textual UCML listing that follows represents the canonical model produced by the reference editor.

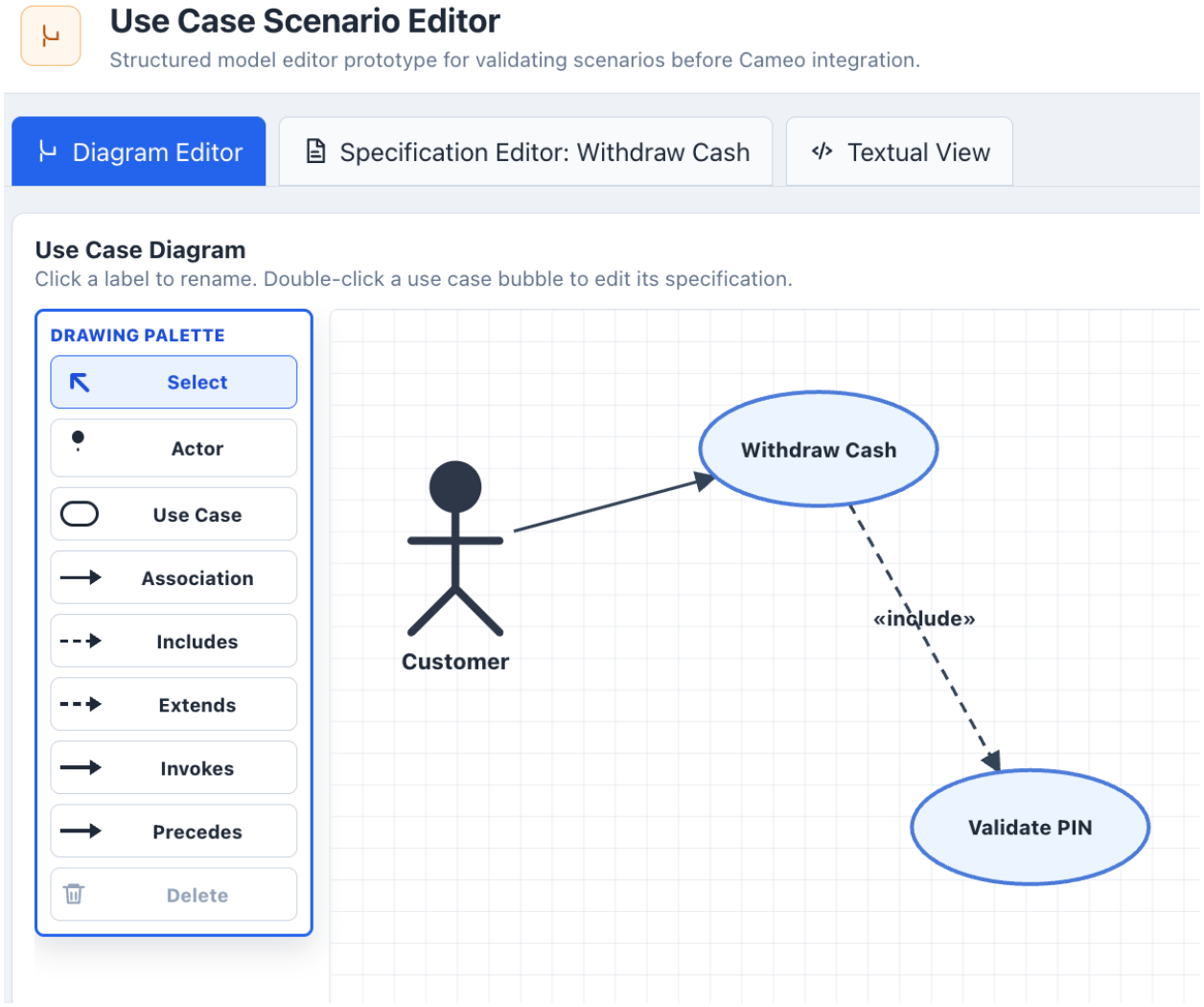



Figure 7. Withdraw Cash Use Case Diagram.

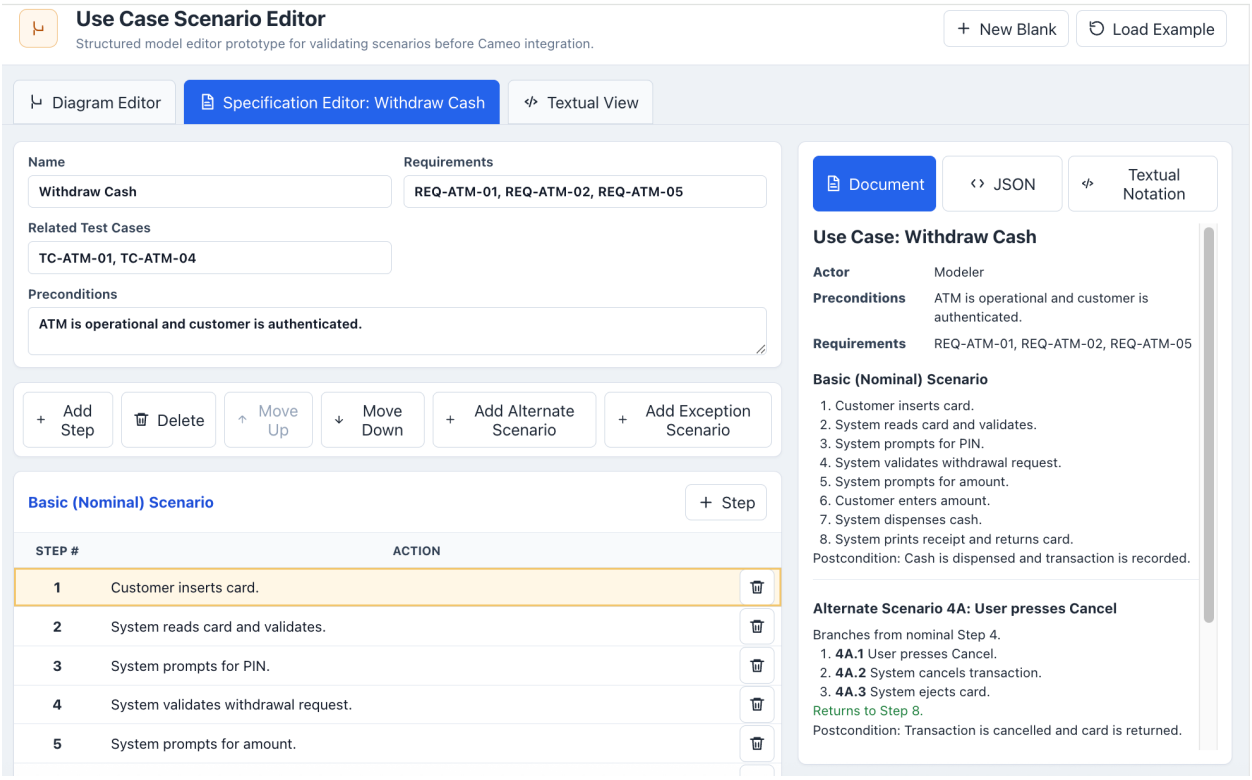



**Figure 8. Withdraw Cash Specification Editor.**

## 5.3 Example UCML Model

The following listing is taken directly from the reference editor. It is intentionally presented verbatim as the canonical example of the proposed UCML textual notation.

```
model aTMUseCaseModel {

  actor customer {
    id: "actor1";
    name: "Customer";
  }

  use case "Withdraw Cash" {
    id: "uc1";

    specification {
      primaryActorId: "actor1";

      requirements {
        "REQ-ATM-01";
        "REQ-ATM-02";
        "REQ-ATM-05";
      }

      relatedTestCases {
        "TC-ATM-01";
        "TC-ATM-04";
      }

      preconditions {
```

```
  "ATM is operational and customer is authenticated.";
}

scenario basic {
  step 1 {
    id: "step-uc1-N-customer-inserts-card";
    text: "Customer inserts card.";
  }

  step 2 {
    id: "step-uc1-N-system-reads-card-and-validates";
    text: "System reads card and validates.";
  }

  step 3 {
    id: "step-uc1-N-system-prompts-for-pin";
    text: "System prompts for PIN.";
  }

  step 4 {
    id: "step-uc1-N-system-validates-withdrawal-request";
    text: "System validates withdrawal request.";
  }

  step 5 {
    id: "step-uc1-N-system-prompts-for-amount";
    text: "System prompts for amount.";
  }

  step 6 {
    id: "step-uc1-N-customer-enters-amount";
    text: "Customer enters amount.";
  }

  step 7 {
    id: "step-uc1-N-system-dispenses-cash";
    text: "System dispenses cash.";
  }

  step 8 {
    id: "step-uc1-N-system-prints-receipt-and-returns-card";
    text: "System prints receipt and returns card.";
  }

  postconditions {
    "Cash is dispensed and transaction is recorded.";
  }
}

scenario alternate 4A {
  branchStepId: "step-uc1-N-system-validates-withdrawal-request";
  condition: "User presses Cancel";
  rejoinStepId: "step-uc1-N-system-prints-receipt-and-returns-card";
  endsUseCase: false;

  step 4A.1 {
    id: "step-uc1-4A-user-presses-cancel";
    text: "User presses Cancel.";
  }

  step 4A.2 {
    id: "step-uc1-4A-system-cancels-transaction";
```

```
          text: "System cancels transaction.";
        }

        step 4A.3 {
          id: "step-uc1-4A-system-ejects-card";
          text: "System ejects card.";
        }

        postconditions {
          "Transaction is cancelled and card is returned.";
        }
      }

      scenario exception 4E {
        branchStepId: "step-uc1-N-system-validates-withdrawal-request";
        condition: "Insufficient account balance";
        rejoinStepId: "step-uc1-N-system-prompts-for-amount";
        endsUseCase: false;

        step 4E.1 {
          id: "step-uc1-4E-system-displays-insufficient-funds-message";
          text: "System displays insufficient funds message.";
        }

        step 4E.2 {
          id: "step-uc1-4E-system-prompts-for-a-smaller-amount";
          text: "System prompts for a smaller amount.";
        }

        postconditions {
          "No cash is dispensed and customer may enter a smaller amount.";
        }
      }
    }
  }

  use case "Validate PIN" {
    id: "uc2";
    specification {
      scenario basic {
        step 1 {
          id: "step-uc2-N-step-1";
          text: "";
        }
      }
    }
  }

  association "edge1" {
    id: "edge1";
    sourceId: "actor1";
    targetId: "uc1";
  }

  include "«include»" {
    id: "edge2";
    sourceId: "uc1";
    targetId: "uc2";
  }
}
```

## 6. Why an Alternative Use Case Library?

This proposal has two objectives: to standardize structured use case specifications through the Structured Use Case Specification Library and to provide the Structured Use Case Diagram Library, which restores simplicity, flexibility, and natural communication while encouraging modeling practices that improve requirements discovery, behavioral completeness, systematic verification, and ultimately more robust systems.

The primary motivation for this proposal is the observation that most engineers construct use case models in essentially the same way. In practice, they typically begin by following a simple three-step process:

1. ***Identify an actor.***
2. ***Identify the goals (use cases) associated with that actor.***
3. ***Connect the actor to those use cases to establish the initial use case diagram.***

This straightforward workflow has proven effective for decades because it naturally organizes system functionality around its users.

The current SysML v2 Diagram Library makes this fundamental modeling workflow unreasonably difficult by modeling Actors as parameters of Use Cases. As a result, representing a single actor associated with multiple use cases becomes unnecessarily complex, even though this is how many engineers begin constructing a use case model. **In effect, it breaks the natural use case modeling workflow at its very first step.**

This illustrates a broader design philosophy that appears in several areas of SysML v2: optimizing for formal representation rather than for the people creating and reading the models. The result is a language that is too often *machine-friendly but human-hostile*. While such representations may simplify tool processing, they impose unnecessary complexity on engineers performing common modeling tasks. The current SysML v2 Use Case Library is a vivid illustration of this philosophy.

The Structured Use Case Diagram Library restores this natural modeling style while preserving the simplicity, flexibility, and natural communication that have traditionally made use case diagrams effective engineering tools.

The Structured Use Case Specification Library addresses an even more important objective. The current SysML v2 use case library does not provide a standardized semantic representation for structured use case specifications, even though they contain the primary engineering value of use case modeling. As a result, the language does not currently encourage the systematic exploration of alternate and exception scenarios that has become established industrial practice. Without a standard semantic representation for these scenarios, opportunities to discover off-nominal requirements,

improve behavioral completeness, and derive comprehensive behavioral tests are diminished.

Together, the Structured Use Case Specification Library and the Structured Use Case Diagram Library comprise the Structured Use Cases Library. The Diagram Library restores the intuitive modeling workflow that engineers have used successfully for decades, while the Specification Library standardizes the primary engineering artifact of use case modeling, encouraging more complete requirements discovery through systematic exploration of alternate and exception scenarios and providing a stronger foundation for comprehensive behavioral testing. Together, they provide a practical alternative that standardizes proven engineering practices while improving interoperability, model quality, and the overall effectiveness of use case modeling.

## 7. Open-Source Reference Editor

To encourage thorough evaluation and rapid adoption of the proposed Structured Use Cases Library, the authors have developed a free-to-use reference editor implementing the Structured Use Case Specification Library and the Structured Use Case Diagram Library described in this proposal. The reference editor will be released as open source software through the OpenMBEE community as part of the Pain-Free Use Cases Project.

The primary purpose of the reference editor is not to compete with commercial MBSE tools, but to provide a practical implementation of the proposed libraries that can be used for experimentation, education, interoperability evaluation, and everyday use. The editor demonstrates that the proposed libraries are practical, implementable, and suitable for real-world use case modeling.

A public landing page for the Structured Use Cases project and hosted reference editor is available at https://www.parallelagile.com/simple-use-cases.html.

Included with the reference editor is the ability to export a textual representation of the underlying metamodel for the current use case model. This notation, called UCML (Use Case Modeling Language), is modeled after the SysML v2 textual notation and provides a concise, human-readable representation of the Structured Use Case Specification Library and the Structured Use Case Diagram Library. Like SysML v2 textual notation, UCML provides a textual view of the underlying model, enabling use case models to be exchanged, version controlled, reviewed, and processed using text-based development workflows.

The reference editor, together with the Structured Use Case Specification Library, the Structured Use Case Diagram Library, and the UCML textual notation, comprise the technical foundation of the Pain-Free Use Cases Project. The project will be released as open source software through the OpenMBEE community and is described in more detail in the following section.

## 8. Open Source Project (Pain-Free Use Cases)

The Pain-Free Use Cases Project is an open source initiative intended to encourage experimentation, community participation, and rapid adoption of the proposed Structured Use Cases Library. The project consists of the Structured Use Case Specification Library, the Structured Use Case Diagram Library, the UCML (Use Case Modeling Language) textual notation, and the open source reference editor introduced in the previous section. The project will be released through the OpenMBEE community, providing a collaborative environment in which researchers, practitioners, tool vendors, and OMG participants can evaluate, extend, and refine the proposed libraries.

The current public project landing page is https://www.parallelagile.com/simple-use-cases.html. The page provides a stable entry point for the hosted reference editor and related Parallel Agile training resources while the OpenMBEE project materials are being prepared.

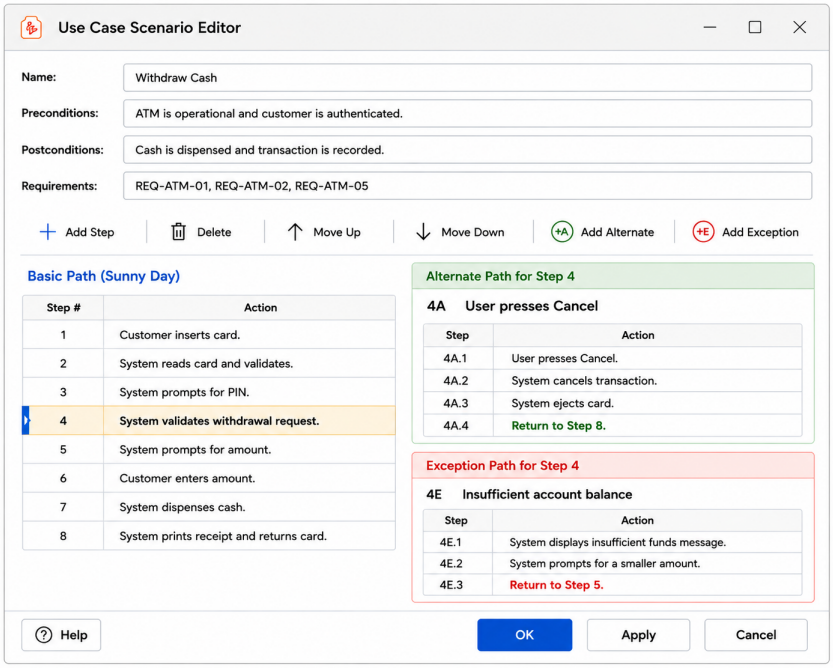



**Figure 9: The Open Source editor will be published as part of Pain-Free Use Cases**

Figure 9 illustrates the current implementation of the Pain-Free Use Cases reference editor. The editor provides a practical environment for developing structured use case specifications using the proposed metamodel. It supports creation and editing of the basic scenario together with alternate and exception scenarios, encouraging systematic exploration of off-nominal behavior while maintaining the semantic relationships defined by the Structured Use Case Specification Library. The editor also provides navigation among scenarios, making it practical to organize and maintain complete behavioral specifications for systems of realistic size.

The reference editor includes a graphical diagram editor for creating Structured Use Case diagrams and structured use case specifications. It also supports exporting the current use case model using UCML (Use Case Modeling Language), a human-readable textual notation modeled after the SysML v2 textual notation. UCML provides a concise textual representation of the underlying model suitable for review, version control, and interchange. In addition, the editor supports saving complete use case models in JSON format, providing a machine-readable representation for tool integration and automated processing.

The reference editor was developed using AI-Assisted MBSE (AIM) to produce the implementation specification. Initial implementation work was performed using Claude Code, with the production implementation completed using OpenAI Codex from the AIM specification. The resulting editor serves as the reference implementation of the proposed Structured Use Cases Library.

By making the **Pain-Free Use Cases Project** freely available through the **OpenMBEE** community, the authors hope to encourage experimentation and community participation while providing a practical reference implementation for the proposed OMG standard. Together, the proposed OMG standard and the **Pain-Free Use Cases Project** provide both a formal semantic foundation and a practical implementation, accelerating evaluation, adoption, and future evolution of structured use case modeling.

## 9. Proposed OMG Standardization

This proposal recommends that OMG adopt the Structured Use Cases Library as an alternative standard use case library for SysML v2. Because SysML v2 is designed to evolve through its libraries, the proposal requires no changes to the SysML v2 language itself. Instead, it extends the standard library with an alternative use case library that the authors believe offers significant advantages over the current use case library, as discussed in Section 6, including a higher value-to-pain ratio, while remaining fully compatible with the SysML v2 architecture.

The proposed standard consists of two complementary libraries: the Structured Use Case Specification Library, which provides a standardized semantic representation for structured use case specifications, and the Structured Use Case Diagram Library, which restores the simplicity, flexibility, and natural communication traditionally associated with use case diagrams. Together, these libraries provide a complete semantic representation of a use case model while preserving compatibility with existing SysML v2 concepts.

The authors recommend that both libraries be standardized as part of the SysML v2 library ecosystem and be made available alongside the existing use case library. This approach allows organizations to adopt the library that best fits their engineering practices while encouraging evaluation, comparison, and continued evolution through practical experience. Existing SysML v2 models remain unaffected, while new projects gain access to a standardized representation of structured use case specifications together with a more flexible and intuitive approach to use case diagrams.

The accompanying Pain-Free Use Cases Project provides a practical reference implementation of the proposed libraries, including the open source reference editor, the UCML textual notation, and supporting software. Together, the proposed OMG standard and the open source project provide both a formal semantic foundation and a practical implementation, encouraging evaluation, adoption, and continued refinement by the broader MBSE community.

The public project landing page at https://www.parallelagile.com/simple-use-cases.html provides a stable URL that OMG reviewers, tool vendors, and community participants can use to access the hosted reference editor during proposal evaluation.

The authors welcome review, discussion, and collaboration from the OMG community and look forward to working with tool vendors, practitioners, and standards participants to refine the proposal and evolve the Structured Use Cases Library into a broadly adopted industry standard.

## 10. Increasing Behavioral Test Coverage

Although improved behavioral testing is not itself the primary objective of this proposal, it illustrates one of the important engineering benefits of standardizing structured use case specifications. By representing system behavior as organized scenarios composed of ordered steps, the proposed metamodel provides a natural semantic foundation for more systematic behavioral testing.

The **Design-Driven Testing** book introduced two complementary forms of testing derived directly from structured use case specifications. The first is **requirement testing**, in which one or more test cases are derived for each requirement to verify that the requirement has been correctly implemented. The second is the **Use Case Thread Expander**, which systematically expands the basic, alternate, and exception scenarios within a use case specification into a complete set of end-to-end behavioral test threads.

The effectiveness of both testing approaches depends directly on the completeness of the underlying requirements. By encouraging systematic exploration of alternate and exception scenarios, the proposed metamodel helps engineers discover additional off-nominal requirements that might otherwise be overlooked. Those newly discovered requirements naturally become candidates for additional requirement tests, while the corresponding scenarios become additional behavioral threads. More complete requirements therefore lead directly to more complete behavioral test coverage.

The proposed metamodel directly supports both testing approaches. Each **Scenario** defines a distinct behavioral path through the system, while the separate **Postcondition** associated with every **Scenario** provides the expected result for that behavioral thread. These scenario-specific postconditions naturally support **assertion testing**, allowing each test case to verify not only that the correct sequence of behavior occurred, but also that the expected outcome of the scenario was achieved.

When **Design-Driven Testing** was published in 2010, comprehensive requirement testing and use case thread expansion were considered too labor-intensive for routine industrial application. Recent advances in AI-assisted, specification-driven development have dramatically reduced this effort. In the authors' experience, AI coding agents such as **Claude Code** and **OpenAI Codex** are particularly effective at generating both assertion tests and use case thread tests from well-structured use case specifications, making these Design-Driven Testing techniques considerably more practical today.

Rather than requiring the expanded behavioral threads to be stored as part of the specification, AI coding agents can execute the thread expansion strategy directly from the structured use case narrative during test generation.

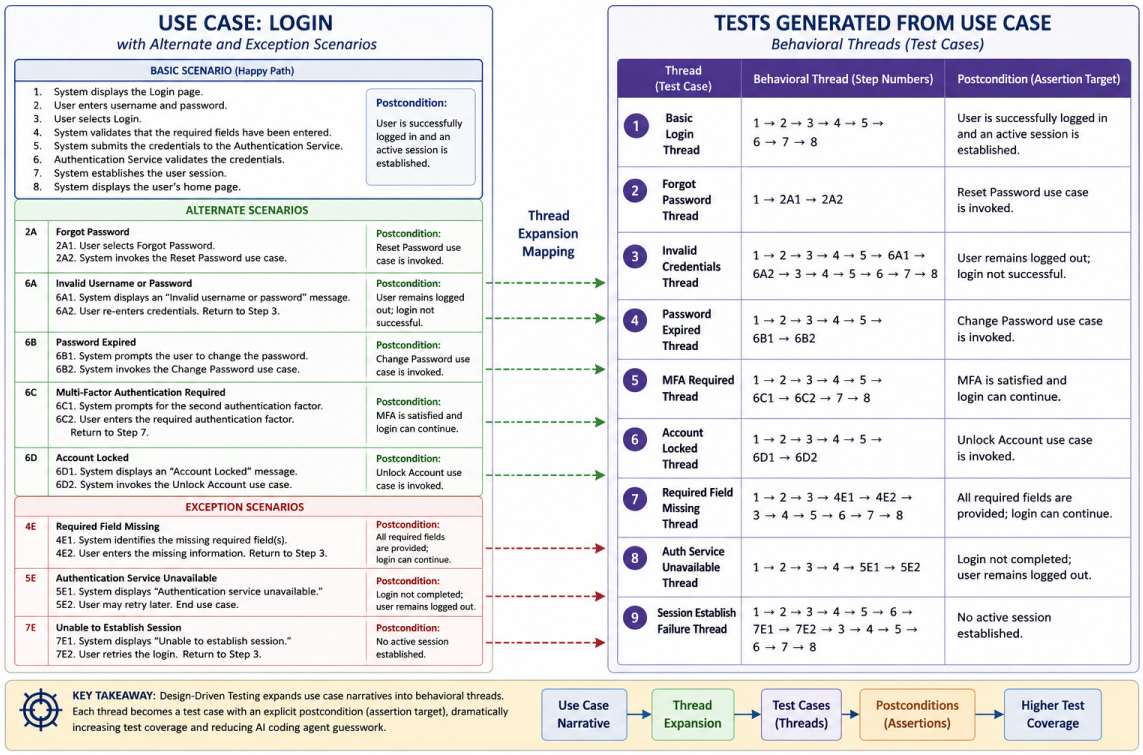



**Figure 10 – Expanding use case threads dramatically improves test coverage**

To summarize, the effectiveness of behavioral testing depends on the completeness of the requirements model, and the completeness of the requirements model depends on systematic exploration of alternate and exception scenarios.

## 11. Conclusions

This proposal has presented the Structured Use Cases Library as an alternative use case library for SysML v2 consisting of two complementary parts: the Structured Use Case Specification Library and the Structured Use Case Diagram Library. Together, these libraries provide a standardized semantic representation for structured use case specifications while restoring the simplicity, flexibility, and natural communication that have traditionally made use case diagrams effective engineering tools.

The proposal begins from a simple observation: the primary engineering value of use case modeling resides in the use case specifications, not in the diagrams themselves. Use case diagrams are most valuable as organizational and communication aids, while the specifications capture the behavioral semantics of the system. In particular, the systematic exploration of alternate and exception scenarios has long been recognized as one of the most effective techniques for discovering missing off-nominal requirements.

The Structured Use Case Specification Library standardizes this established industrial practice by providing a common semantic representation for structured use case specifications based on use cases, scenarios, and steps. Standardizing this representation promotes interoperability among modeling tools while encouraging more complete behavioral specifications, improved requirements discovery, and a stronger foundation for behavioral verification.

The Structured Use Case Diagram Library restores the straightforward modeling workflow that engineers have used successfully for decades. Rather than imposing unnecessary restrictions on how use case diagrams are constructed, it enables modelers to organize system functionality naturally around actors and their goals. By making diagrams simple and flexible while placing the engineering emphasis on the accompanying specifications, the proposed library provides a significantly higher value-to-pain ratio for everyday use case modeling.

The proposal is complemented by the Pain-Free Use Cases Project, an open source reference implementation consisting of the proposed libraries, the UCML textual notation, and a free reference editor. Together, the proposed OMG standard and the open source implementation provide both a formal semantic foundation and a practical environment for evaluation, experimentation, and community participation through the OpenMBEE community.

Finally, the proposal demonstrates that standardized structured use case specifications provide benefits extending well beyond documentation. By encouraging systematic exploration of alternate and exception scenarios, the proposed metamodel promotes more complete requirements discovery while providing a natural semantic foundation for requirement testing, assertion testing, use case thread expansion, and

comprehensive behavioral verification. Recent advances in AI-assisted, specification-driven development make these techniques considerably more practical than when they were first introduced, further increasing the value of standardized behavioral specifications.

The authors believe the Structured Use Cases Library represents a practical evolution of use case modeling that reflects established industrial practice while remaining fully compatible with the SysML v2 architecture. By standardizing structured use case specifications, restoring a simple and intuitive diagramming approach, and providing an open source reference implementation, the proposal offers the OMG community an opportunity to improve interoperability, increase the value-to-pain ratio of use case modeling, and encourage broader adoption of proven engineering practices.